Отчёт по лабораторной работе №4

Имитационное моделирование

Самарханова Полина Тимуровна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Выполнение лабораторной работы **7**

3.1 Описание модели 7
3.2 Базовый эксперимент 9
3.3 Сканирование коэффициента заразности 10
3.4 Исследование эффекта миграции 12
3.5 Многокритериальная оптимизация параметров 13
3.6 Сводная визуализация результатов 15

4 Выводы **18**

Список иллюстраций

3.1 создание файла 7
3.2 код 8
3.3 установка пакетов 8
3.4 загрузка 8
3.5 модель 9
3.6 файл 9
3.7 код 9
3.8 установка пакетов 10
3.9 график 10
3.10 файл 10
3.11 код 11
3.12 установка пакетов 11
3.13 график и данные 12
3.14 файл 12
3.15 код 12
3.16 результат 13
3.17 файл 13
3.18 код 14
3.19 код 14
3.20 код 15
3.21 файл 15
3.22 код 16
3.23 код 17

Список таблиц

1 Цель работы

Целью данной лабораторной работы является изучить и реализовать агентную SIR-модель распространения эпидемии и провести исследования.

2 Задание

- Создать рабочий каталог для всего курса. -Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Выполнение лабораторной работы

3.1 Описание модели

Перехожу в каталог лабораторной работы 4 и создаю папку src и файл sir_model.jl (рис. 3.1).

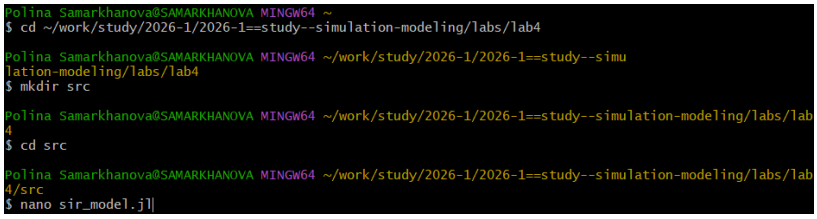

Рисунок 3.1: создание файла

В открывшемся окне вставила код из методички (рис. 3.2).

```
  GNU nano 8.7                                   sir_model.jl
# 4.1.3 Кон Модель

# – src/sir_model.jl

# 139

# Реализация основных моделей в агентном подходе

using Agents, Random
using Agents.Graphs
using StatsBase: sample, Weights
using Distributions: Poisson
using DrWatson: @dict

# Определение типа агента
@agent struct Person(GraphAgent)
    days_infected::Int  # количество дней с момента заражения
    status::Symbol      # :S, :I, :R
end

# Функция инициализации модели
function initialize_sir(;
    Ns = [1000, 1000, 1000],        # численность по трём городам
    migration_rates = nothing,       # матрица миграции
    B_und = [0.5, 0.5, 0.5],         # передача невыявленными
    B_det = [0.05, 0.05, 0.05],      # передача выявленными
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [5, 5, 1],                  # начальное количество инфицированных
    seed = 42,
    n_steps = 100,
)
    rng = Xoshiro(seed)
    C = length(Ns)  # количество городов

    # Если матрица миграции не задана, создаем реалистичную
    if migration_rates === nothing
        migration_rates = zeros(C, C)
        for i = 1:C
            for j = 1:C
                # Вероятность миграции пропорциональна размеру целевого города
                migration_rates[i, j] = (Ns[i] + Ns[j]) / Ns[i]
            end
        end
        # Нормализация
        for i = 1:C
            migration_rates[i, :] ./= sum(migration_rates[i, :])
        end
    end

    properties = @dict(
        Ns,
                                        [ Read 174 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
```

Рисунок 3.2: код

Установила необходимые пакеты (рис. 3.3).

```
julia> import Pkg

julia> Pkg.add(["Agents", "Random", "StatsBase", "Distributions", "DrWatson", "Graphs"])
```

Рисунок 3.3: установка пакетов

Загружаем модель (рис. 3.4).

```
julia> include("src/sir_model.jl")
total_count (generic function with 1 method)
```

Рисунок 3.4: загрузка

Создала экземпляр модели(рис. 3.5).

```
julia> model = initialize_sir()
StandardABM with 3000 agents of type Person
 agents container: Dict
 space: GraphSpace with 3 positions and 3 edges
 scheduler: fastest
 properties: B_det, C, infection_period, Ns, migration_rates, detection_time, B_und, reinfection_probab
lity, death_rate
```

Рисунок 3.5: модель

3.2 Базовый эксперимент

Создала папки data, plots, scripts и файл sir_run_basic.jl (рис. 3.6).

```
julia> import Pkg

julia> Pkg.add(["JLD2", "DrWatson"])
   Resolving package versions...
   Installed JLD2 — v0.6.4
    Updating `C:\Users\Polina Samarkhanova\.julia\environments\v1.12\Project.toml`
  [033835bb] + JLD2 v0.6.4
    Updating `C:\Users\Polina Samarkhanova\.julia\environments\v1.12\Manifest.toml`
  [033835bb] ↑ JLD2 v0.6.3 ⇒ v0.6.4
Precompiling packages finished.
  5 dependencies successfully precompiled in 115 seconds. 388 already precompiled.
```

Рисунок 3.6: файл

В открывшемся окне вставила код из медодички (рис. 3.7).

```
  GNU nano 8.7                                scripts/sir_run_basic.jl
total_vals = Int[]

# Запуск симуляции вручную
for step = 1:params[:n_steps]
    Agents.step!(model, 1)

    push!(times, step)
    push!(S_vals, susceptible_count(model))
    push!(I_vals, infected_count(model))
    push!(R_vals, recovered_count(model))
    push!(total_vals, total_count(model))
end

# Создаём DataFrame
agent_df = DataFrame(time = times, susceptible = S_vals, infected = I_vals, recov
model_df = DataFrame(time = times, total = total_vals)

# Визуализация
plot(
    agent_df.time,
    agent_df.susceptible,
    label = "Восприимчивые",
    xlabel = "Дни",
    ylabel = "Количество",
)
plot!(agent_df.time, agent_df.infected, label = "Инфицированные")
plot!(agent_df.time, agent_df.recovered, label = "Выздоровевшие")
plot!(agent_df.time, model_df.total, label = "Всего (включая умерших)", linestyle
savefig(joinpath("plots", "sir_basic_dynamics.png"))

# Сохранение данных
using JLD2: @save
@save joinpath("data", "sir_basic_agent.jld2") agent_df
@save joinpath("data", "sir_basic_model.jld2") model_df
```

Рисунок 3.7: код

Установила необходимые пакеты (рис. 3.8).

```
julia> import Pkg

julia> Pkg.add(["JLD2", "DrWatson"])
   Resolving package versions...
   Installed JLD2 — v0.6.4
    Updating `C:\Users\Polina Samarkhanova\.julia\environments\v1.12\Project.toml`
  [033835bb] + JLD2 v0.6.4
    Updating `C:\Users\Polina Samarkhanova\.julia\environments\v1.12\Manifest.toml`
  [033835bb] ↑ JLD2 v0.6.3 ⇒ v0.6.4
Precompiling packages finished.
  5 dependencies successfully precompiled in 115 seconds. 388 already precompiled.
```

Рисунок 3.8: установка пакетов

Запустила код и получила результат(график) (рис. 3.9).

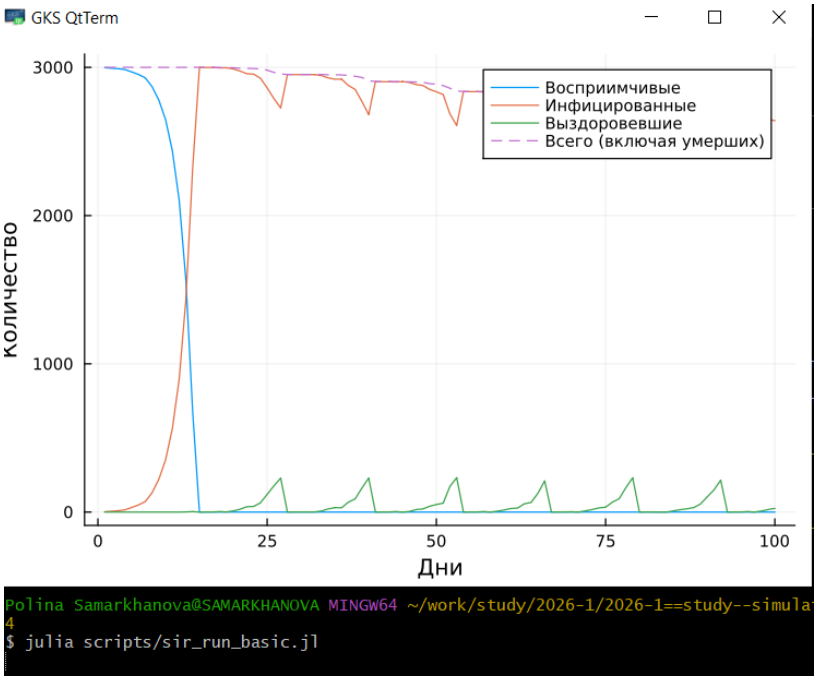

Рисунок 3.9: график

3.3 Сканирование коэффициента заразности

Создала файл scripts/sir_scan_beta.jl (рис. 3.10).

```
Polina Samarkhanova@SAMARKHANOVA MING
4
$ nano scripts/sir_scan_beta.jl
```

Рисунок 3.10: файл

В открывшемся окне вставила код из медодички (рис. 3.11).

```
  GNU nano 8.7                                scripts/sir_scan_beta.jl
using Statistics
grouped = combine(
    groupby(df, [:beta]),
    :peak => mean => :mean_peak,
    :final_inf => mean => :mean_final_inf,
    :deaths => mean => :mean_deaths,
)

# График
plot(
    grouped.beta,
    grouped.mean_peak,
    label = "Пик эпидемии",
    xlabel = "Коэффициент заразности β",
    ylabel = "Доля инфицированных",
    marker = :circle,
    linewidth = 2,
)
plot!(
    grouped.beta,
    grouped.mean_final_inf,
    label = "Конечная доля инфицированных",
    marker = :square,
)
plot!(grouped.beta, grouped.mean_deaths ./ 3000, label = "Доля умерших", marker = :diamond)
savefig(joinpath("plots", "beta_scan.png"))

println("Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png")
```

Рисунок 3.11: код

Установила необходимые пакеты (рис. 3.12).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulatio
4
$ julia
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.12.5 (2026-02-09)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org release
|__/                   |

(@v1.12) pkg> ]add CSV
ERROR: `]add` is not a recognized command. Type ? for help with available commands

(@v1.12) pkg> add CSV
   Resolving package versions...
    Updating `C:\Users\Polina Samarkhanova\.julia\environments\v1.12\Project.toml`
  [336ed68f] + CSV v0.10.16
    Manifest No packages added to or removed from `C:\Users\Polina Samarkhanova\.juli
2\Manifest.toml`

(@v1.12) pkg>
```

Рисунок 3.12: установка пакетов

Запустила код и получила результат (рис. 3.13).

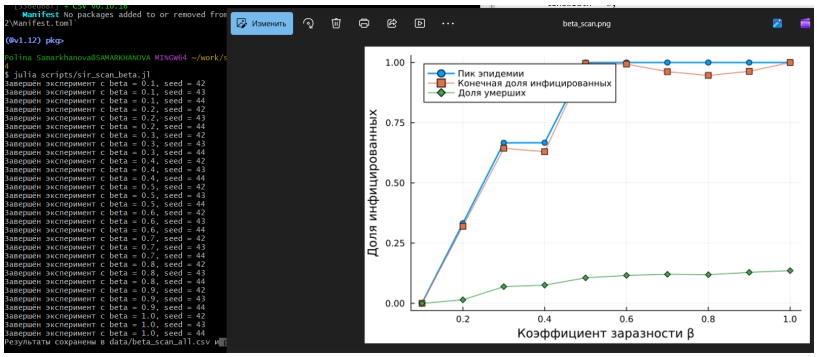

Рисунок 3.13: график и данные

3.4 Исследование эффекта миграции

Создала файл sir_migration_effect.jl (рис. 3.14).

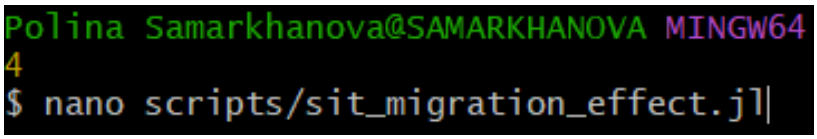

Рисунок 3.14: файл

В открывшемся окне вставила код из медодички (рис. 3.15).

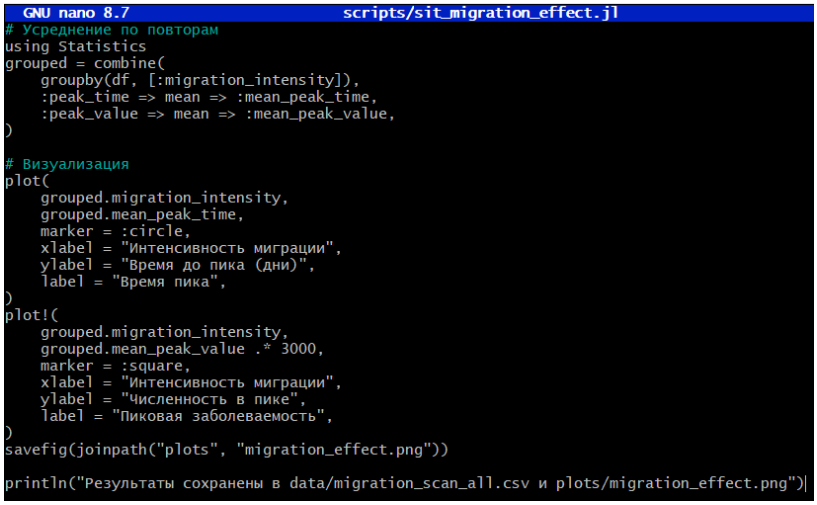

Рисунок 3.15: код

Запустила код и получила результат (рис. 3.16).

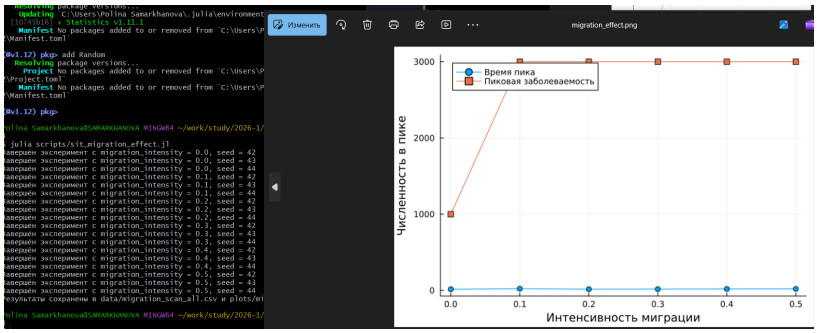

Рисунок 3.16: результат

3.5 Многокритериальная оптимизация параметров

Создала файл sir_optimize_parameters.jl(рис. 3.17).

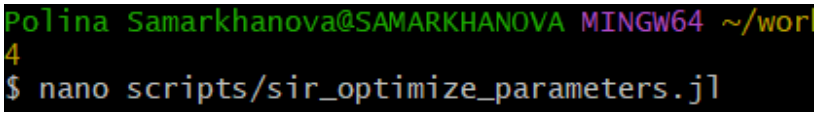

Рисунок 3.17: файл

В открывшемся окне вставила код из медодички (рис. 3.18).

```
MINGW64:/c/Users/Polina Samarkhanova/work/study/2026-1/2026-1==study--simulation-modeling/labs/lab4
  GNU nano 8.7                    scripts/sir_optimize_parameters.jl
# scripts/sir_optimize_parameters.jl

using BlackBoxOptim, Random, Statistics
using DrWatson
using JLD2

# Подключаем модель
include(joinpath(@__DIR__, "..", "src", "sir_model.jl"))

# Целевая функция: минимизируем пиковую заболеваемость и смертность
function cost_multi(x)
    # x[1]: β_und, x[2]: detection_time, x[3]: death_rate
    replicates = 5
    peak_vals = Float64[]
    dead_vals = Int[]

    for rep = 1:replicates
        model = initialize_sir(
            Ns = [1000, 1000, 1000],
            B_und = fill(x[1], 3),
            B_det = fill(x[1]/10, 3),
            infection_period = 14,
            detection_time = round(Int, x[2]),
            death_rate = x[3],
            reinfection_probability = 0.1,
            Is = [0, 0, 1],
            seed = 42 + rep,
        )

        infected_frac(model) = count(a.status == :I for a in allagents(model)) / nagents(mode
        dead_count(model) = 3000 - nagents(model)

        peak_infected = 0.0

        for step = 1:100
            Agents.step!(model, 1)
            frac = infected_frac(model)
            if frac > peak_infected
```

Рисунок 3.18: код

Установила необходимые пакеты (рис. 3.19).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulat
4
$ julia
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.12.5 (2026-02-09)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org release
|__/                   |

(@v1.12) pkg> add BlackBoxOptim
   Resolving package versions...
   Installed SpatialIndexing — v0.1.6
   Installed BlackBoxOptim ─── v0.6.4
    Updating `C:\Users\Polina Samarkhanova\.julia\environments\v1.12\Project.toml`
  [a134a8b2] + BlackBoxOptim v0.6.4
    Updating `C:\Users\Polina Samarkhanova\.julia\environments\v1.12\Manifest.toml`
  [a134a8b2] + BlackBoxOptim v0.6.4
  [d4ead438] + SpatialIndexing v0.1.6
Precompiling packages finished.
  2 dependencies successfully precompiled in 13 seconds. 393 already precompiled.
```

Рисунок 3.19: код

Запустила код и получиларезультат (рис. 3.20).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/lab
4
$ julia scripts/sir_optimize_parameters.jl
Starting optimization with optimizer BlackBoxOptim.BorgMOEA{EpsBoxDominanceFitnessScheme{2, Float64, tru
e, typeof(sum)}, BlackBoxOptim.ProblemEvaluator{Tuple{Float64, Float64}, IndexedTupleFitness{2, Float64}
, EpsBoxArchive{2, Float64, EpsBoxDominanceFitnessScheme{2, Float64, true, typeof(sum)}}, FunctionBasedP
roblem{typeof(cost_multi), ParetoFitnessScheme{2, Float64, true, typeof(sum)}, ContinuousRectSearchSpace
, Nothing}}, FitPopulation{IndexedTupleFitness{2, Float64}}, FixedGeneticOperatorsMixture, RandomBound{C
ontinuousRectSearchSpace}}
0.00 secs, 0 evals, 0 steps
pop.size=50 arch.size=0 n.restarts=0

Optimization stopped after 1 steps and 646.24 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.08
Improvements/step = NaN
Total function evaluations = 51


Best candidate found: [0.20144, 3.01581, 0.021027]

Fitness: (0.00073, 0.00000) agg=0.00073


==================================================
РЕЗУЛЬТАТЫ ОПТИМИЗАЦИИ
==================================================
Оптимальные параметры:
  β_und = 0.2014397837660047
  Время выявления = 3 дней
  Смертность = 0.021026951210328184

Достигнутые показатели:
  Пик заболеваемости: 0.0007333333333333333
  Доля умерших: 0.0
==================================================

Результаты сохранены в data/optimization_result.jld2
```

Рисунок 3.20: код

3.6 Сводная визуализация результатов

Создала файл sir_visualize_dynamics.jl(рис. 3.21).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/wo
4
$ nano scripts/sir_visualize_dynamics.jl
```

Рисунок 3.21: файл

В открывшемся окне вставила код из медодички (рис. 3.22).

```
GNU nano 8.7                    scripts/sir_visualize_dynamics.jl
# scripts/sir_visualize_dynamics.jl

using DrWatson
using Agents, DataFrames, Plots, CSV

# Подключаем модель (для вспомогательных функций, если нужны)
include(joinpath(@__DIR__, "..", "src", "sir_model.jl"))

# Загружаем результаты сканирования
df = CSV.read(joinpath("data", "beta_scan_all.csv"), DataFrame)

# Создаём составной график
p1 = plot(df.beta, df.peak, label = "Пик", xlabel = "β", ylabel = "Доля инфицированных")
plot!(p1, df.beta, df.final_inf, label = "Конечная")

p2 = plot(df.beta, df.deaths, xlabel = "β", ylabel = "Число умерших")

p3 = plot(df.beta, df.final_rec, xlabel = "β", ylabel = "Доля выздоровевших")

plot(p1, p2, p3, layout = (3, 1), size = (800, 900))
savefig(joinpath("plots", "comprehensive_analysis.png"))

println("График сохранён в plots/comprehensive_analysis.png")
```

Рисунок 3.22: код

Запустила код и получиларезультат (рис. 3.23).

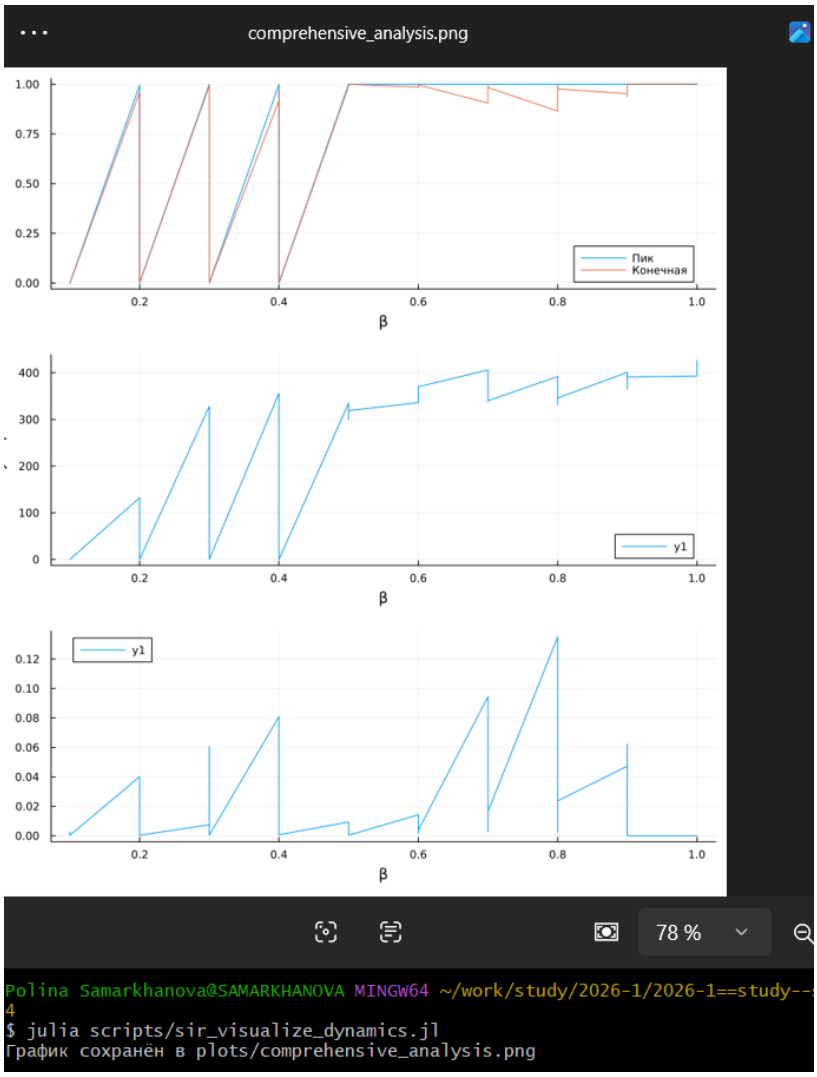

Рисунок 3.23: код

4 Выводы

В ходе выполнения лабораторной работы была успешно реализована и протестирована агентная SIR-модель с пространственной структурой(3 города и миграция).